MODUL PENGAJAR DAN MAHASISWA

Pengembangan Situs Web I

Minggu 9: JavaScript Modern: Data, Kontrol Alur, dan Function

Materi lengkap pendamping Sesi 1 - konsep, contoh, diagnosis, dan pengujian

Kode mata kuliah: 4141103
Program Studi D4 Teknologi Rekayasa Perangkat Lunak
Tahun akademik 2026/2027

Petunjuk penggunaan

Modul ini ditujukan kepada mahasiswa tingkat I. Baca penjelasan sebelum mencoba contoh. Gunakan presentasi untuk diskusi, modul ini untuk bacaan lebih lengkap, dan modul Sesi 2-3 untuk pengerjaan terarah.

> Prasyarat
> Gunakan hasil pekerjaan sendiri dari minggu sebelumnya. Minggu 9 meneruskan fondasi yang sudah dipelajari, tanpa React atau backend.

- CPL tetap mengikuti RPS hasil review. Tujuan di dalam modul adalah tujuan kegiatan, bukan perubahan kode CPL/CPMK resmi.
- Contoh memakai data fiktif; jangan memasukkan akun CIS atau data pribadi nyata.
- Kode pendek merupakan potongan yang perlu ditempatkan sesuai penjelasan. Struktur HTML minimum mengikuti Minggu 2.
- Simpan file, uji dengan browser, catat hasil, dan jelaskan alasan perubahan.

Tujuan dan alur kegiatan

- Menjelaskan let, const, tipe data, dan operator.
- Menulis conditional serta loop yang sederhana.
- Mengolah array/object dengan function.
- Mengenali scope, destructuring, module, dan pemeriksaan kode.

Waktu	Kegiatan Sesi 1
0-5 menit	Pemantik dan pemeriksaan prasyarat
5-20 menit	Konsep utama beserta contoh
20-35 menit	Demonstrasi serta prediksi hasil
35-45 menit	Diagnosis dan diskusi kasus
45-50 menit	Ringkasan dan exit ticket

Praktikum menggunakan dua sesi terpisah, masing-masing dengan alokasi latihan 120 menit yang dapat disesuaikan jadwal laboratorium.

1. JavaScript dan cara menjalankan

JavaScript memberi perilaku pada halaman. Pada minggu ini fokus pada data dan logika; DOM dipelajari Minggu 10. Kode dapat dicoba di Console lalu disimpan dalam file. Gunakan script defer untuk file biasa agar eksekusi menunggu parsing HTML, atau type module untuk module. Console membantu eksperimen, tetapi pekerjaan akhir harus dapat dijalankan ulang dari file sumber. Jangan menempelkan kode tidak dikenal pada halaman akun pribadi.

Contoh dan pengamatan

```
<script src="main.js" defer></script>
// main.js
console.log("Latihan JavaScript");
```

Pertanyaan pemahaman
Apa beda eksperimen Console dan sumber yang dapat dijalankan ulang?

2. let dan const

let digunakan ketika binding akan diberi nilai lain. const digunakan ketika binding tidak akan dipindahkan ke nilai lain. const tidak membuat isi object atau array otomatis tidak berubah. Pilih const sebagai dasar dan let ketika memang perlu reassignment. Nama variabel harus menjelaskan data, seperti jumlahPeserta. Jangan memakai satu variabel untuk arti yang berubah-ubah.

Contoh dan pengamatan

```
const nama = "Rina";
let jumlah = 1;
jumlah = jumlah + 1;
const peserta = [];
peserta.push("Rina");
```

Pertanyaan pemahaman
Mengapa push pada array const tetap dapat dilakukan?

3. Tipe data dan konversi

String menyimpan teks, number menyimpan angka, boolean menyimpan benar/salah, sedangkan null dan undefined memiliki arti yang berbeda. Nilai dari banyak input form berupa string. Konversi dengan Number bila kebutuhan memang angka; periksa Number.isFinite sebelum menghitung. Jangan menganggap tampilan 2 berarti data sudah number. typeof null menghasilkan object karena perilaku historis; itu tidak berarti null adalah object biasa.

Contoh dan pengamatan

```
const teks = "2";
const angka = Number(teks);
console.log(typeof teks, typeof angka);
console.log(Number.isFinite(angka));
```

Pertanyaan pemahaman
Apa hasil "2" + 1 dibanding Number("2") + 1?

4. Operator dan expression

Expression menghasilkan nilai. Operator aritmetika menghitung, operator perbandingan membentuk boolean. Gunakan === dan !== ketika ingin perbandingan tanpa coercion. Operator + dapat menjumlahkan atau menggabungkan string, sehingga tipe perlu diperhatikan. Template literal membantu menyusun teks dengan nilai. Hindari mengandalkan konversi implisit ketika hasil perlu dijelaskan oleh mahasiswa.

Contoh dan pengamatan

```
const tiket = 2;
const harga = 10000;
const total = tiket * harga;
console.log(`Total: ${total}`);
console.log(2 === "2");
```

Pertanyaan pemahaman
Mengapa === membantu menghindari perbandingan membingungkan?

5. Conditional dan validasi sederhana

if menjalankan cabang berdasarkan kondisi. Gunakan urutan yang jelas: periksa nilai tidak valid dahulu, lalu jalankan proses. else if membantu membedakan beberapa rentang. Kondisi merupakan expression boolean. Jangan membuat cabang yang tidak mungkin tercapai karena cabang sebelumnya terlalu luas. Uji batas tepat pada minimum dan maksimum, bukan hanya satu data yang diterima.

Contoh dan pengamatan

```
const jumlah = Number("3");
if (!Number.isInteger(jumlah) || jumlah < 1 || jumlah > 5) {
 console.log("Jumlah harus bilangan bulat 1-5");
} else { console.log("Jumlah diterima"); }
```

Pertanyaan pemahaman

Apa hasil untuk 0, 1, 5, 6, dan 1.5?

6. Loop dan penghentian

Loop mengulangi pekerjaan. for of nyaman untuk membaca setiap nilai array. for dengan indeks berguna ketika posisi diperlukan. while membutuhkan kondisi yang dapat berubah agar berhenti. Jangan membuat loop tanpa batas pada browser karena halaman dapat macet. Mulai dari data kecil dan tulis harapan jumlah iterasi. break menghentikan loop, sedangkan continue melanjutkan ke iterasi berikutnya.

Contoh dan pengamatan

```
const kegiatan = ["HTML", "CSS", "Audit"];
for (const nama of kegiatan) {
 console.log(nama);
}
```

Pertanyaan pemahaman

Berapa kali loop berjalan untuk array kosong?

7. Function, parameter, dan return

Function membungkus pekerjaan yang mempunyai tujuan jelas. Parameter menerima input; return memberi hasil kepada pemanggil. console.log menampilkan tetapi tidak menggantikan return. Function yang mengembalikan data mudah diuji tanpa halaman. Pisahkan perhitungan dari tampilan. Nama hitungTotal lebih jelas daripada proses karena menunjukkan hasil yang diharapkan.

Contoh dan pengamatan

```
function hitungTotal(jumlah, harga) {
 return jumlah * harga;
}
console.log(hitungTotal(2, 10000));
```

Pertanyaan pemahaman

Apa yang terjadi jika function hanya console.log tetapi tidak return?

8. Array dan pengolahan koleksi

Array menyimpan urutan data dan memakai indeks mulai nol. map membentuk array hasil transformasi, filter memilih elemen, dan find mengambil kecocokan pertama atau undefined. Pilih sesuai kebutuhan; jangan memakai map hanya untuk efek samping tanpa hasil. push menambah item dan mengubah array. Uji array kosong karena sering menjadi kondisi normal ketika pencarian tidak menemukan data.

Contoh dan pengamatan

```
const angka = [1, 2, 3];
const duaKali = angka.map(n => n * 2);
const besar = angka.filter(n => n >= 2);
console.log(duaKali, besar);
```

Pertanyaan pemahaman

Apa beda hasil filter dan find?

9. Object dan data terstruktur

Object mengelompokkan properti terkait, misalnya judul, kuota, dan status suatu kegiatan. Akses dengan titik ketika nama diketahui dan bracket ketika nama properti berasal dari nilai lain. Data array object menjadi dasar katalog pada minggu berikutnya. Jangan memasukkan struktur dengan properti berbeda tanpa pemeriksaan. Gunakan nama serta tipe yang konsisten agar function dapat mengolahnya.

Contoh dan pengamatan

```
const kegiatan = { id: 1, judul: "HTML", kuota: 20 };
console.log(kegiatan.judul);
const kolom = "kuota";
console.log(kegiatan[kolom]);
```

Pertanyaan pemahaman

Mengapa konsistensi nama properti penting?

10. Scope dan destructuring

Scope menentukan tempat nama dapat diakses. let dan const mempunyai block scope. Variabel dalam function tidak otomatis tersedia di luar function. Destructuring mengambil properti atau elemen ke binding baru. Gunakan hanya ketika membuat kode lebih jelas, bukan sebagai tujuan mempersingkat semua baris. Hindari variabel global berlebihan karena bagian kode lain dapat memengaruhinya tanpa sengaja.

Contoh dan pengamatan

```
const kegiatan = { judul: "HTML", kuota: 20 };
const { judul, kuota } = kegiatan;
if (kuota > 0) { const pesan = "Tersedia"; }
// pesan tidak tersedia di luar block
```

Pertanyaan pemahaman

Apa manfaat membatasi scope?

11. Module dasar

Module memisahkan kode dengan export dan import. Script type module memiliki scope tersendiri dan berjalan dengan aturan module. Jalankan melalui server lokal agar permintaan file tidak terganggu batasan file URL. Path import relatif harus benar. Minggu ini cukup satu module utilitas dan satu main, tanpa bundler atau framework. Jangan mewajibkan npm hanya untuk contoh module sederhana.

Contoh dan pengamatan

```
// utils.js
export function hitungTotal(n, harga) { return n * harga; }
// main.js
import { hitungTotal } from "./utils.js";
console.log(hitungTotal(2, 10000));
```

Pertanyaan pemahaman

Mengapa contoh module dijalankan lewat server lokal?

12. Defensive coding dan linting dasar

Defensive coding memeriksa asumsi sebelum memakai data. Periksa array, string, atau angka yang dibutuhkan agar pesan gagal jelas. Linter memeriksa pola kode dan sebagian potensi kesalahan, tetapi tidak membuktikan logika benar. Gunakan pemeriksaan editor yang tersedia; konfigurasi ESLint hanya opsional sesuai laboratorium. Uji function dengan input normal, batas, dan invalid serta baca Console.

Contoh dan pengamatan

```
function hitungTotal(n, harga) {
 if (!Number.isInteger(n) || n < 1) return null;
 if (!Number.isFinite(harga) || harga < 0) return null;
 return n * harga;
}
```

Pertanyaan pemahaman

Mengapa hasil linter bersih belum membuktikan perhitungan benar?

Demonstrasi dosen dan diskusi kasus

Gunakan langkah-langkah berikut sebagai demonstrasi terpilih. Mahasiswa memprediksi hasil sebelum kode dijalankan. Dosen meminta bukti browser dan penjelasan setelah percobaan, lalu membandingkan hasil dengan prediksi.

File dan Console
Buat HTML minimum dan main.js. Hubungkan script defer.
Harapan: File dieksekusi tanpa SyntaxError.

Konversi data
Eksperimen string dan number.
Harapan: Mahasiswa menjelaskan concatenation dan penjumlahan.

Aturan jumlah tiket
Buat pemeriksaan jumlah 1-5 dan harga tetap.
Harapan: Nilai tidak sah tidak dihitung sebagai total normal.

Loop koleksi
Tampilkan daftar judul menggunakan for of.
Harapan: Jumlah keluaran sesuai panjang array.

Penghubung praktikum
Pindahkan perhitungan menjadi function dan siapkan array object untuk Sesi 3.

Rancangan pengujian yang dapat diulang

Tabel ini merupakan acuan harapan, bukan laporan hasil yang sudah dikerjakan. Mahasiswa mencatat hasil aktual dan bukti pada laporan praktikum.

Kasus	Hasil yang diharapkan
String 2 + 1	21
Number 2 + 1	3
Jumlah 0/6	Ditolak
Array kosong	Tidak ada iterasi
Function valid	Total benar
Filter semua nol	Array kosong
Path import salah	Error terdeteksi
Scope block	Binding tidak tersedia di luar

- Catat browser, ukuran viewport, dan kondisi jika memengaruhi hasil.
- Untuk kasus gagal, tulis pesan atau gejala, sumber masalah, serta perbaikannya.
- Setelah perbaikan, jalankan kasus yang sama dan kasus regresi.
- Jangan menandai belum diimplementasikan sebagai lulus.

Ringkasan, refleksi, dan tugas persiapan

- Menjelaskan let, const, tipe data, dan operator.
- Menulis conditional serta loop yang sederhana.
- Mengolah array/object dengan function.
- Mengenali scope, destructuring, module, dan pemeriksaan kode.

Pertanyaan refleksi

- Apa beda eksperimen Console dan sumber yang dapat dijalankan ulang?
- Apa hasil "2" + 1 dibanding Number("2") + 1?
- Apa hasil untuk 0, 1, 5, 6, dan 1.5?
- Apa yang terjadi jika function hanya console.log tetapi tidak return?
- Mengapa konsistensi nama properti penting?
- Mengapa contoh module dijalankan lewat server lokal?

> Persiapan berikutnya
> Siapkan halaman dengan daftar dan form untuk interaksi DOM Minggu 10.

Exit ticket: tuliskan konsep yang sudah dipahami, satu bukti pengujian yang dapat dijelaskan, dan satu pertanyaan yang masih perlu didiskusikan.

Lampiran contoh integrasi 1

index.html - script module, tidak memakai script defer bersamaan

Contoh ini merupakan acuan setelah latihan, bukan pengganti proses pengerjaan. Bila label menyebut body, masukkan ke dokumen HTML minimum. Bila file JavaScript terbagi beberapa bagian, gabungkan berurutan dalam satu file tanpa mengulang deklarasi.

```
<!doctype html>
<html lang="id">
<head>
 <meta charset="utf-8">
 <meta name="viewport" content="width=device-width, initial-scale=1">
 <title>Latihan JavaScript</title>
</head>
<body>
 <h1>Latihan JavaScript</h1>
 <p>Buka Console untuk hasil dan jalankan melalui server lokal.</p>
 <script type="module" src="main.js"></script>
</body>
</html>
```

Verifikasi

Jalankan melalui server lokal. Gunakan matriks uji di modul dan periksa Console. Untuk HTML/CSS, uji keyboard, viewport, dan zoom. Jangan mencampur isi file JSON dengan HTML/JavaScript.

Lampiran contoh integrasi 2

utils.js

Contoh ini merupakan acuan setelah latihan, bukan pengganti proses pengerjaan. Bila label menyebut body, masukkan ke dokumen HTML minimum. Bila file JavaScript terbagi beberapa bagian, gabungkan berurutan dalam satu file tanpa mengulang deklarasi.

```
export function hitungTotal(jumlah, harga) {
 if (!Number.isInteger(jumlah) || jumlah < 1 || jumlah > 5) {
  return null;
 }
 if (!Number.isFinite(harga) || harga < 0) return null;
 return jumlah * harga;
}
```

Verifikasi
Jalankan melalui server lokal. Gunakan matriks uji di modul dan periksa Console. Untuk HTML/CSS, uji keyboard, viewport, dan zoom. Jangan mencampur isi file JSON dengan HTML/JavaScript.

Lampiran contoh integrasi 3

main.js

Contoh ini merupakan acuan setelah latihan, bukan pengganti proses pengerjaan. Bila label menyebut body, masukkan ke dokumen HTML minimum. Bila file JavaScript terbagi beberapa bagian, gabungkan berurutan dalam satu file tanpa mengulang deklarasi.

```
import { hitungTotal } from "./utils.js";
const kegiatan = [
 { id: 1, judul: "HTML", kuota: 20 },
 { id: 2, judul: "CSS", kuota: 0 },
 { id: 3, judul: "Audit", kuota: 10 }
];
const tersedia = kegiatan.filter(k => k.kuota > 0);
console.log(tersedia.map(k => k.judul));
for (const jumlah of [0, 1, 5, 6, 1.5]) {
 console.log(jumlah, hitungTotal(jumlah, 10000));
}
console.assert(hitungTotal(2, 10000) === 20000);
console.assert(hitungTotal(0, 10000) === null);
console.assert(hitungTotal(6, 10000) === null);
```

Verifikasi

Jalankan melalui server lokal. Gunakan matriks uji di modul dan periksa Console. Untuk HTML/CSS, uji keyboard, viewport, dan zoom. Jangan mencampur isi file JSON dengan HTML/JavaScript.

Referensi dan sumber contoh

MDN - JavaScript Guide
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

MDN - JavaScript Modules
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules

W3C WAI - Forms Tutorial
https://www.w3.org/WAI/tutorials/forms/

W3C - WCAG 2.2
https://www.w3.org/TR/WCAG22/

WHATWG - HTML Living Standard
https://html.spec.whatwg.org/

Contoh dan rancangan alur disusun khusus untuk paket ini berdasarkan referensi resmi. Tidak ada gambar ilustratif pihak ketiga di dalam modul. Template identitas/logo mengikuti dokumen praktikum sebelumnya. Tanggal penyusunan dan pemeriksaan sumber: 15 September 2026.

Gunakan dokumentasi resmi untuk detail API. Materi project merupakan usulan panduan pelaksanaan: ukuran tim, bobot penilaian, dan jadwal penyerahan resmi tetap mengikuti keputusan dosen.